A2_2

Array Sum - 100,000,000 elements					
	Sequential	1 Thread	2 Threads	3 Threads	4 Threads
Time	.305s	.608s	.565s	.561s	.573s

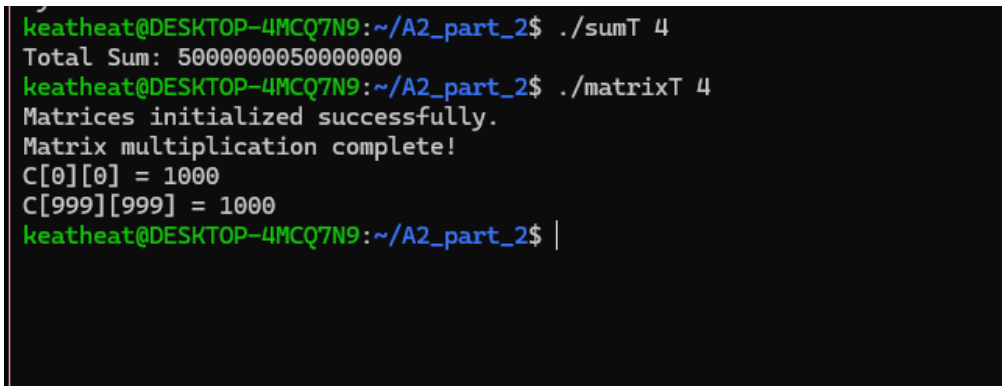
Matrix Multiplication - 1000 x 1000					
	Sequential	1 Thread	2 Threads	3 Threads	4 Threads
Time	-	0.553s	.287s	.197s	.146s

Array Sum

The array sum did not benefit much from multithreading. The sequential version took .305 seconds, while the parallel versions took between .561 and .608 seconds. The extra thread management overhead was greater than the benefit of splitting up the relatively simple work. No synchronization primitive was needed because each thread works on a separate section of the array and its own partial sum

Matrix Multiplication

Matrix multiplication benefited much more from multithreading. The time decreased from .553 seconds with 1 thread to .146 seconds with 4 threads because the larger amount of work can be divided among the threads. No synchronization primitive was needed because each thread works on different rows of the result matrix

Manual synchronization

The manual approach works because the work is divided into independent sections. Each thread modifies its own data, and pthread_join is used to make sure all threads finish before the final results are used.